ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

HONORIS UNITED UNIVERSITIES

Compte Rendu
Développement Front-End

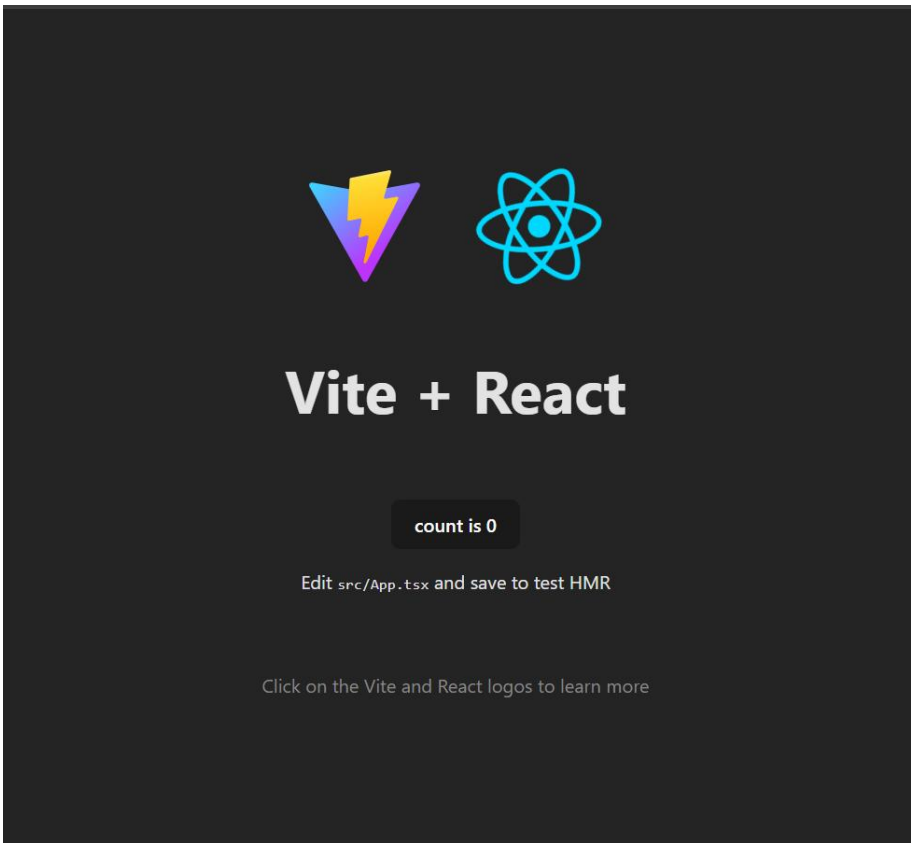
TP N° 1
Le web moderne et react

Réalisée par : Bitit Hind

Classe : 4DDSIR 5

Partie 1 : Initialisation du projet

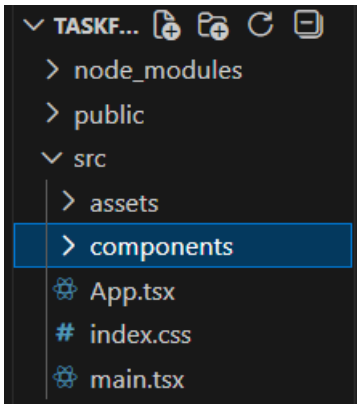
1.1 Créer le projet React avec Vite



1.2 Nettoyage

src > # index.css > ...

```
1
2   * {
3     margin: 0;
4     padding: 0;
5     box-sizing: border-box;
6   }
7
8   html,
9   body,
10  #root {
11    height: 100%;
12    font-family: 'Inter', system-ui, sans-serif;
13    background: #f0f0f0;
14  }
```



4. Que contient le <body> ? Lien avec le CSR ?

```
index.html > html > body
1  <!doctype html>
2  <html lang="en">
3    <head>
4      <meta charset="UTF-8" />
5      <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7      <title>taskflow</title>
8    </head>
9    <body>
10     <div id="root"></div>
11     <script type="module" src="/src/main.tsx"></script>
12   </body>
13 </html>
14
```

Le <body> contient :
Le div est le point de montage de l'application React.
Le fichier main.tsx injecte le composant <App> dans ce div.

Lien avec le CSR (client side render)
Le navigateur charge une page HTML presque vide, puis React génère dynamiquement toute l'interface côté client grâce au JavaScript.

Partie 2 : Backend avec json-server

2.2 Créer db.json

```
db.json > ...
1  {
2    "projects": [
3      { "id": "1", "name": "Site E-commerce", "color": "#e74c3c" },
4      { "id": "2", "name": "App Mobile", "color": "#3498db" },
5      { "id": "3", "name": "API Backend", "color": "#2ecc71" }
6    ],
7    "columns": [
8      { "id": "todo", "title": "À Faire", "tasks": ["Maquettes UI", "Specs techniques"] },
9      { "id": "progress", "title": "En Cours", "tasks": ["API Auth", "Page Login"] },
10     { "id": "done", "title": "Terminé", "tasks": ["Setup projet", "Config ESLint"] }
11   ]
12 }
```

2.3 Lancer le serveur

```
PS C:\Users\pc\taskflow> json-server --watch db.json --port 4000
Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:4000/projects
http://localhost:4000/columns
```

localhost:4000/projects

Impression élégante


```
[
  {
    "id": "1",
    "name": "Site E-commerce",
    "color": "#e74c3c"
  },
  {
    "id": "2",
    "name": "App Mobile",
    "color": "#3498db"
  },
  {
    "id": "3",
    "name": "API Backend",
    "color": "#2ecc71"
  }
]
```


localhost:4000/columns

Impression élégante


```
[
  {
    "id": "todo",
    "title": "À Faire",
    "tasks": [
      "Maquettes UI",
      "Specs techniques"
    ]
  },
  {
    "id": "progress",
    "title": "En Cours",
    "tasks": [
      "API Auth",
      "Page Login"
    ]
  },
  {
    "id": "done",
    "title": "Terminé",
    "tasks": [
      "Setup projet",
      "Config ESLint"
    ]
  }
]
```


Différence entre données en dur et API REST

Les données en dur sont écrites directement dans le code, ne sont pas dynamiques et nécessitent modifier le code pour changer les informations.

Une API REST fournit les données depuis un serveur via des requêtes http, les données sont dynamiques, séparées du frontend et plus proches d'une vraie application professionnelle.

Partie 3 : Composants avec Props

3.1 *Pourquoi className au lieu de class en JSX ?*

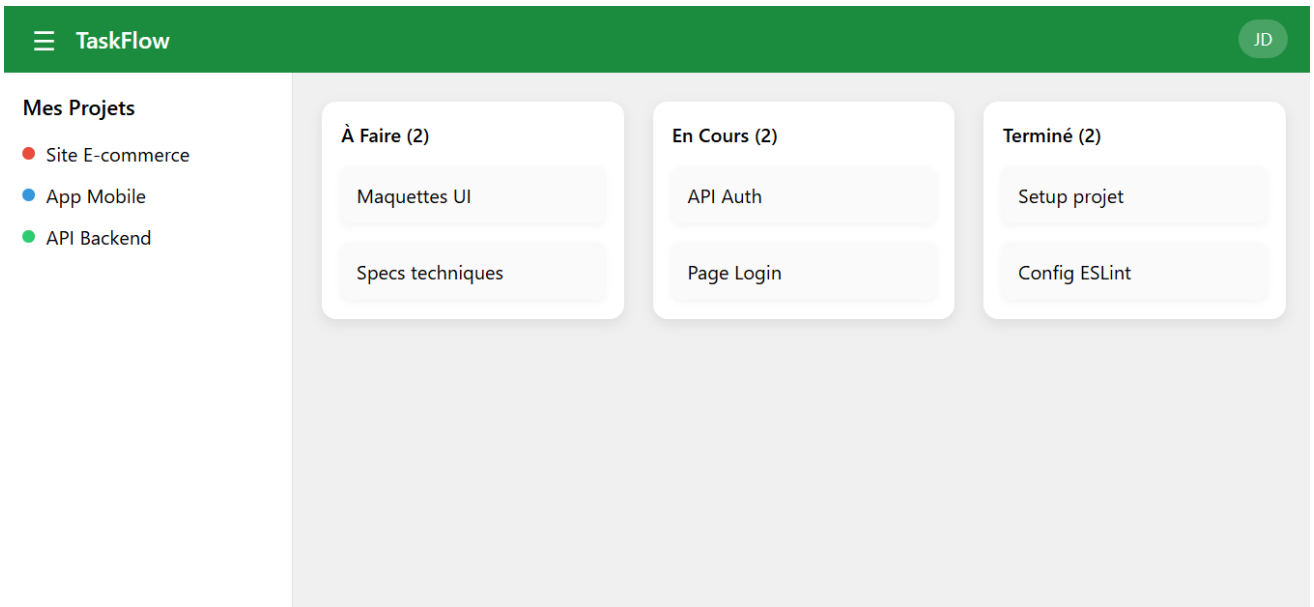
En JSX, on utilise className car class est un mot réservé en JavaScript.

3.2 *Pourquoi key={p.id} est obligatoire dans .map() ? Que se passe-t-il avec l'index ?*

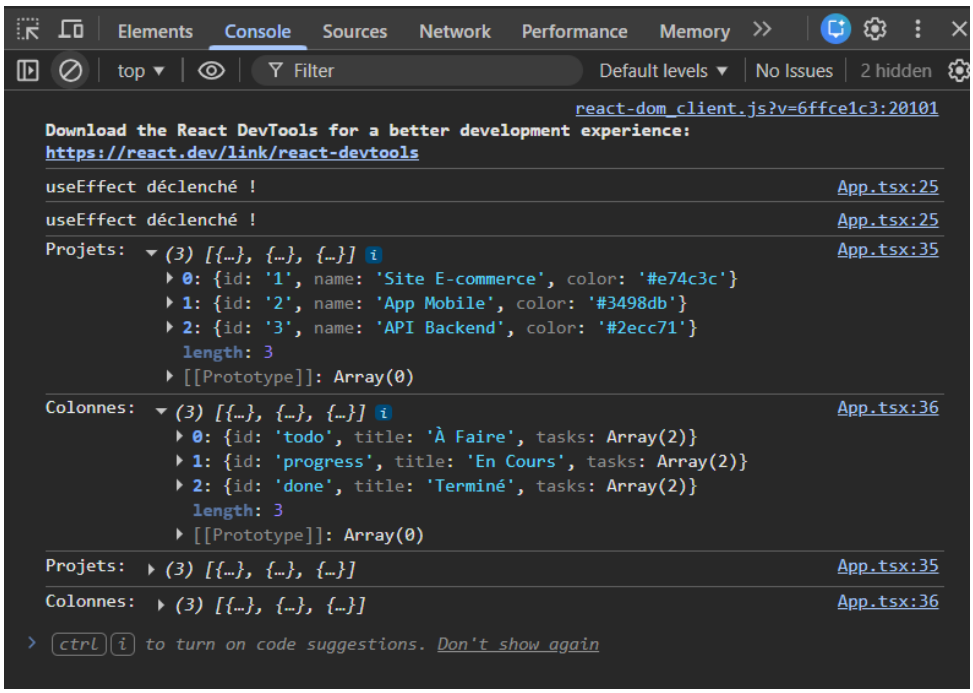
La prop key permet à React d'identifier de manière unique chaque élément d'une liste.Elle aide React à optimiser le re-render et à savoir quel élément a été modifié.

Partie 4 : State, useEffect & Fetch

4.1 App.tsx — charger depuis l'API



4.2 Vérifier et comprendre



Q1 : Combien de fois le useEffect s'exécute-t-il ? Pourquoi ?

➤ Une seule fois, car le tableau [] est vide.

Q2 : Arrêtez json-server (Ctrl+C) et rechargez. Que se passe-t-il ?

➤fetch échoue → catch déclenche console.error, les composants restent, mais sans données.

```
Elements  Console  Sources  Network  Performance  »   6
top ▾   Filter   Default levels ▾   No Issues   2 hidden
                                                  react-dom_client.js?v=6ffce1c3:20101
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
useEffect déclenché !                                   App.tsx:25
useEffect déclenché !                                   App.tsx:25
▶GET http://localhost:4000/projects net::ERR_CONNECTION_REFUSED   App.tsx:29
▶Erreur: TypeError: Failed to fetch                     App.tsx:40
    at fetchData (App.tsx:29:11)
    at App.tsx:46:5
  Show ignore-listed frames
▶GET http://localhost:4000/columns net::ERR_CONNECTION_REFUSED    App.tsx:30
▶GET http://localhost:4000/projects net::ERR_CONNECTION_REFUSED   App.tsx:29
▶Erreur: TypeError: Failed to fetch                     App.tsx:40
    at fetchData (App.tsx:29:11)
    at App.tsx:46:5
  Show ignore-listed frames
▶GET http://localhost:4000/columns net::ERR_CONNECTION_REFUSED    App.tsx:30
> ctrl i to turn on code suggestions. Don't show again
```

Q3 : Ouvrez Network (F12). Voyez-vous les requêtes vers localhost:4000 ? Code HTTP ?

➤ Oui

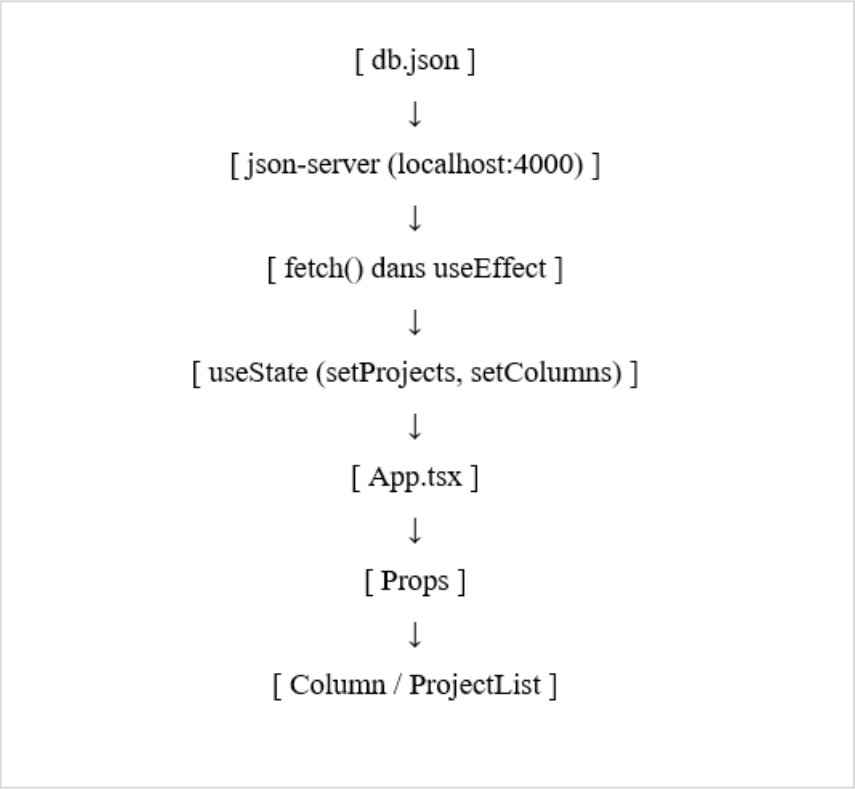
```
Elements  Console  Sources  Network  Performance  Memory  Application  Privacy and security  »
Preserve log   Disable cache   No throttling
localhost:4000                                   Invert   More filters ▾
All  Fetch/XHR  Doc  CSS  JS  Font  Img  Media  Manifest  Socket  Wasm  Other
50 ms  100 ms  150 ms  200 ms  250 ms  300 ms  350 ms  400 ms

Name                 Headers  Preview  Response  Initiator  Timing
projects             ▾ General
columns              Request URL          http://localhost:4000/projects
projects             Request Method       GET
columns              Status Code          ● 200 OK
                     Remote Address       [::1]:4000
                     Referrer Policy      strict-origin-when-cross-origin
                     ▾ Response headers   ☐ Raw
                     Access-Control-Allow-Headers   content-type
                     Access-Control-Allow-Methods   GET, HEAD, PUT, PATCH, POST, DELETE
                     Access-Control-Allow-Origin    *
                     Connection           keep-alive
                     Content-Length       227
                     Content-Type         application/json
                     Date                 Sat, 28 Feb 2026 19:31:42 GMT
                     Keep-Alive           timeout=5
                     X-Powered-By         tinyhttp
                     ▾ Request Headers    ☐ Raw
                     Accept               */*
                     Accept-Encoding      gzip, deflate, br, zstd
4 / 24 requests  2.5 kB / 4.9 kB tra
```

4.3 Tester la réactivité

Modifiez db.json : ajoutez un projet ou une tâche. Rechargez React.

➤ *Q4 : Les nouvelles données s'affichent ? Décrivez le cycle complet.*

Oui, après modification de db.json et rechargement de la page

- json-server met à jour les données
- React recharge la page
- useEffect s'exécute
- fetch() récupère les nouvelles données
- setProjects et setColumns mettent à jour le state
- React re-render les composants
- Les nouvelles données apparaissent à l'écran

➤ *Q5 : Dessinez le flux*